

Spring Boot Advanced

Vorstellung

Philipp Moisel

Principal Consultant

>15 Jahre Java

~7 Jahre Spring Boot

Berater, Entwickler, Architekt, Interviewer, Trainer



Voraussetzungen

- Grundkenntnisse in Spring (Boot)
- Inversion of Control
- Annotations

Agenda

1. Was wollen wir uns anschauen?
2. Security
3. Profiles
4. Validation
5. ApplicationContext
6. Autoconfiguration
7. Actuator
8. Deployment

Was wollen wir uns anschauen?



Ablauf

- Wir erweitern gemeinsam eine kleine Webanwendung
- Einführung ins Thema, dann kleine Aufgabe
- Fragen immer direkt
- Eigene Themen?

Anwendung

- Webanwendung um eine Kundenliste zu verwalten
- Spring MVC mit Thymeleaf

E-Mail	Vorname	Nachname
bill.gates@microsoft.com	Bill	Gates
elon.musk@paypal.com	Elon	Musk
jeff.bezos@amazon.com	Jeff	Bezos

E-Mail:

Vorname:

Nachname:

[zurück](#)

<https://github.com/pmoisel/springbootadvanced>

Security



Story

Als Nutzer mit der Rolle MANAGER möchte ich Kunden anlegen können. Nutzer mit der Rolle USER sollen die Kundenliste sehen dürfen, aber keine Kunden anlegen.

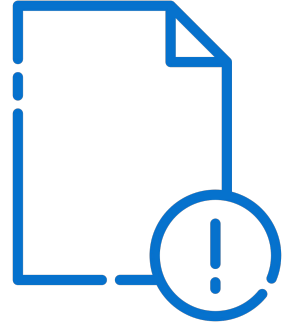
Doku

[Username/Password Authentication](#)

[Authorize Requests](#)

[Method Security](#)

[Hierarchical Roles](#)



Technik

Annotations:

```
@EnableWebSecurity
@EnableMethodSecurity

@PreAuthorize("hasRole('USER')")
@PreAuthorize("hasRole('MANAGER')")
```

Dependencies:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>org.thymeleaf.extras</groupId>
  <artifactId>thymeleaf-extras-springsecurity6</artifactId>
</dependency>
```

Nutzerbasis:

```
@Bean
public InMemoryUserDetailsManager
userDetailsService() {

    return new InMemoryUserDetailsManager(
        User.withDefaultPasswordEncoder()
            .username("user")
            .password("password")
            .roles("USER")
            .build(),
        User.withDefaultPasswordEncoder()
            .username("manager")
            .password("password")
            .roles("MANAGER")
            .build());
}
```

Technik

Role Hierarchy:

```
@Bean
static MethodSecurityExpressionHandler methodSecurityExpressionHandler(RoleHierarchy roleHierarchy) {
    DefaultMethodSecurityExpressionHandler expressionHandler= new DefaultMethodSecurityExpressionHandler();
    expressionHandler.setRoleHierarchy(roleHierarchy);
    return expressionHandler;
}

@Bean
static RoleHierarchy roleHierarchy() {
    return RoleHierarchyImpl.withDefaultRolePrefix()
        .role("MANAGER").implies("USER")
        .build();
}
```

Zusammenfassung



- Rollenbasierte Authentifizierung
- Prüfung pro Methode
- Rollen können Hierarchien abbilden

Profiles



Story

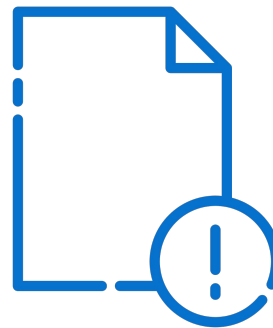
Als Anwender möchte ich die Kundendaten in einer Datenbank persistiert wissen.

Als Entwickler der Anwendung möchte ich diese weiterhin ohne Datenbank starten und weiterentwickeln können.

Doku

Profiles

Initialize a Database



Technik

JpaCustomerRepository:

```
@Repository
@Profile("jpa")
public interface JpaCustomerRepository extends CrudRepository<Customer, String> {}
```

InMemoryCustomerRepository:

```
@Repository
@Profile("!jpa")
public class InMemoryCustomerRepository implements CrudRepository<Customer, String> {}
```

data.sql:

```
INSERT INTO customer (email, first_name, last_name) VALUES ('bill.gates@microsoft.com', 'Bill', 'Gates');
INSERT INTO customer (email, first_name, last_name) VALUES ('elon.musk@paypal.com', 'Elon', 'Musk');
INSERT INTO customer (email, first_name, last_name) VALUES ('jeff.bezos@amazon.com', 'Jeff', 'Bezos');
```

Ziel

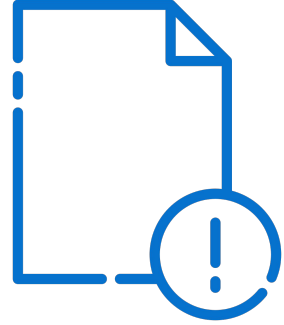
- Nutzer per OAuth2 authentifizieren
- *Keycloak als IdentityProvider*

Doku

[Login Users In with OAuth2](#)

[Common OAuth2 Provider](#)

- google, facebook, okta, **github**



Technik

application.yaml:

```
spring:
  security:
    oauth2:
      client:
        registration:
          github:
            clientId: Ov23lifMeAgDs4Gnkkys
            clientSecret: 61080dd83583f5db98946dc26e9360d964566bf0
```

Security:

```
@Bean
@Profile("oauth")
protected SecurityFilterChain filterChainOAuth(HttpSecurity http) throws Exception {
    http.oauth2Login(Customizer.withDefaults());
    http.logout(logoutConfigurer -> logoutConfigurer
        .logoutSuccessUrl("/index?loggedout"));
    return http.build();
}
```

Dependencies:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-client</artifactId>
</dependency>
```

Zusammenfassung



- Einzelne Teile der Anwendung können mittels Profilen (de-)aktiviert werden
- Datenbanken können automatisch initialisiert werden
- OAuth2 ist leicht zu integrieren
- Trennung zwischen Authentifizierung und Autorisierung

Validation



Story

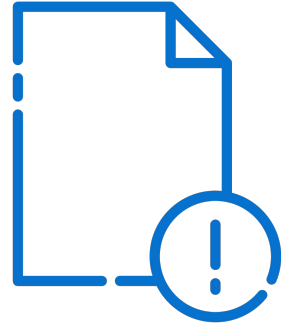
Als Anwender möchte ich, dass meine Eingaben validiert werden. Sollte die Validierung fehlschlagen, so möchte ich entsprechende Meldungen angezeigt bekommen.

Wenn ich neue Kunden anlege, so darf ich das nicht mit einer Amazon-Mail-Adresse.

Doku

[Java Bean Validation](#)

[Customizing Validation Errors](#)



Technik

Bean:

```
public class CustomerDTO {  
  
    @Email  
    private String email;  
  
    @Size(min = 1, max = 40)  
    private String firstName;  
  
    @Size(min = 1, max = 40)  
    private String lastName;  
  
}
```

Dependencies:

```
<dependency>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-validation</artifactId>  
</dependency>
```

Technik

Controller:

```
@PostMapping(value = "/customers", produces = MediaType.TEXT_HTML_VALUE)
@PreAuthorize("hasRole('MANAGER')")
public ModelAndView customerList(
    @Valid @ModelAttribute("newCustomer") CustomerDTO customerFormData,
    BindingResult bindingResult,
    ModelAndView modelAndView) {
    if (!bindingResult.hasErrors()) {
        customerService.save(customerFormData);
        customerFormData = new CustomerDTO();
    }
    modelAndView.addObject("customers", customerService.findAll().toList());
    modelAndView.addObject("newCustomer", customerFormData);
    return modelAndView;
}
```

messages.properties:

Size.newCustomer.firstName=L\u00E4nge muss zwischen {2} und {1} liegen

Technik

Annotation & Validator:

```
@Documented
@Constraint(validatedBy = TrustedDomainValidator.class)
@Target({ElementType.METHOD, ElementType.FIELD})
@Retention(RetentionPolicy.RUNTIME)
public @interface TrustedDomain {
    String message() default "{email.domain}";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}
```

```
public class TrustedDomainValidator implements ConstraintValidator<TrustedDomain, String> {
    @Override
    public boolean isValid(String value, ConstraintValidatorContext context) {
        if (value == null || value.isEmpty()) {
            return true;
        }
        return !value.contains("amazon.com");
    }
}
```

messages.properties:

```
TrustedDomain.email=Bei Amazon gibts nur Jeff!
```

Zusammenfassung



- Spring bietet viele Default-Validatoren
- Errormessages können angepasst werden
- Eigene Validatoren können hinzugefügt werden

ApplicationContext

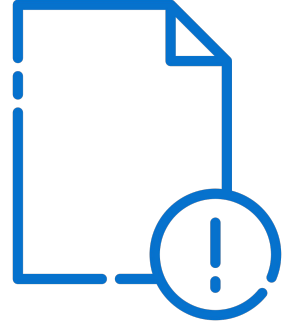


Inhalt

- Bean-Loading
- Abhängigkeiten
- Debugging

Doku

Introduction to the Spring IoC Container and Beans



Technik

@Component

@Controller

@Service

@Repository

@Configuration

@RestController

@SpringBootConfiguration

@SpringBootApplication

Technik

@Bean

@Order

@Qualifier

@Primary

@ConditionalOn*

Technik

- Mehrere Beans vom gleichem Typ
 - Priorities
 - Order
 - Qualifier
- Mehrere Beans vom gleichem Typ injecten
- Nützlich bei Autoconfiguration

Zusammenfassung



- Component-Scanning ab Package der `@SpringBootApplication`-Klasse
- `ApplicationContext` enthält alle Spring-Beans
- Bean-Erstellung kann an Konditionen geknüpft sein
- Autoconfiguration erklärt die Spring-Boot-“Magic”

Autoconfiguration



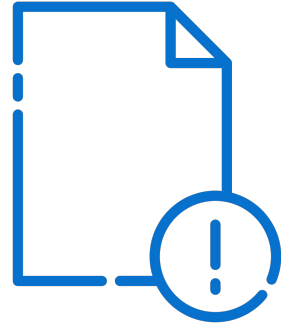
Inhalt

- Verständnis der Spring-Boot-“Magic”
- Eigene Autoconfiguration erstellen

Doku

[Auto-configuration](#)

[Creating You Own Auto-configuration](#)



Technik

Annotation:

```
@AutoConfiguration
public class
CustomObjectMapperAutoconfiguration{

    @Bean
    public ObjectMapper objectMapper() {
        ObjectMapper objectMapper = new
        ObjectMapper();
        objectMapper.disable
        (DeserializationFeature.FAIL_ON_UNKNOW
        N PROPERTIES
        );

        return objectMapper;
    }
}
```

META-INF/spring/org.springframework.boot.autoconfigure.
figure.AutoConfiguration.imports

com.nterra.springbootadvanced.autoconfigure.CustomObjectMappe
rAutoconfiguration

Zusammenfassung



- Autoconfiguration erklärt die Spring-Boot-“Magic”

Actuator



Inhalt

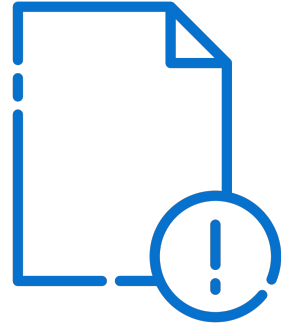
- Metriken für Monitoring bereitstellen
- Liveness- und Readinessprobes für k8s anbieten

Doku

[Production-ready Features](#)

[Metrics](#)

[Health Information](#)



Technik

Dependency:

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-actuator</artifactId>  
</dependency>
```

Zusammenfassung



- Monitoring out-of-the-box
- Einfach Konfiguration für Kubernetes
- Tracing, Observability

Deployment



Inhalt

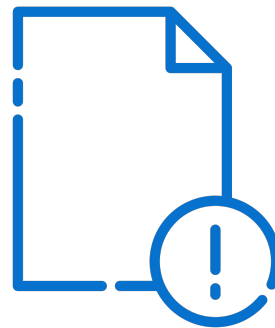
- Anwendung laufen lassen
- Docker-Image erstellen

Doku

[Packaging Executable Archives](#)

[Layered Jar or War](#)

[Packaging OCI Images](#)



Technik

pom.xml:

```
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
      <executions>
        <execution>
          <goals>
            <goal>build-image-no-fork</goal>
          </goals>
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>
```

Zusammenfassung



- offene Diskussion

Weitere Themen

- Testcontainer
- Reactive
- CronJobs



Philipp Moisel
philipp.moisel@nterra.com





Die Expert*innen für Integration

 www.nterra.com